

```
$ ./init-cicd.sh
```

CI/CD con GitHub para equipos (e incluso 1 dev)

Profesionaliza tu flujo sin sobre-ingeniería: del pre-commit al deploy.

github.com/jhoel0521

CI Pipeline · #128

✓	lint & format	12s
✓	unit tests	48s
✓	build	1m 04s
✓	e2e · playwright	3m 12s
●	deploy · production	running

La ruta de hoy

Seis bloques. Cada uno deja algo que podés aplicar hoy mismo en tu repo.

01

El guardián local

Husky, lint-staged y commits que no ensucian el repo.

02

CI en la nube

GitHub Actions: anatomía, niveles de test y velocidad.

03

Ramas y protección

GitFlow simplificado y reglas que te salvan de vos mismo.

04

Trazabilidad

Issues como memoria externa del proyecto.

05

Orquestación

Encadenar jobs y workflows sin duplicar YAML.

06

Cierre

Checklist accionable por semanas.

¿Por qué CI/CD si sos 1 solo dev?

El CI/CD no es un lujo corporativo. Es lo que evita que tu yo del futuro odie a tu yo de hoy.



Sin CI/CD

- Los errores llegan directo a producción y te enterás por el usuario.
- Se pierde el contexto: commits tipo "fix" sin historia detrás.
- Rompés main un viernes y el lunes no sabés qué cambió.



Con CI/CD

- Red de seguridad automática antes de cada merge.
- Historial trazable: cada cambio tiene intención y prueba.
- Delegás las decisiones aburridas a la máquina.

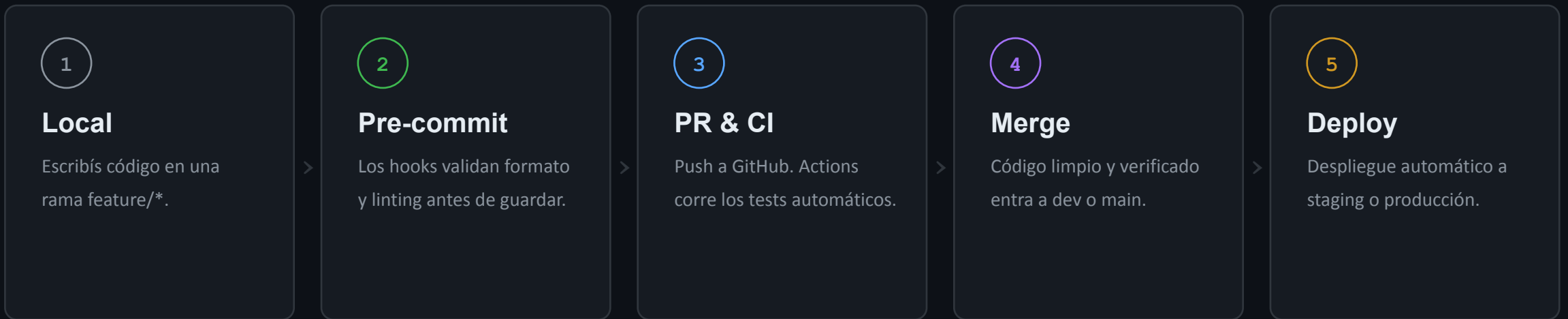
"El CI/CD no es para equipos grandes. Es para no romper cosas."

Mapa del flujo completo

Cinco etapas. Las dos primeras pasan en tu máquina; el resto lo corre GitHub por vos.

TU MÁQUINA

LA NUBE DE GITHUB



Regla de oro: cuanto más temprano falla algo, más barato es arreglarlo. El pre-commit cuesta 2 segundos; el hotfix en producción cuesta tu domingo.

01

SECCIÓN

El guardián local

Todo lo que se puede frenar antes de llegar a Git, hay que frenarlo antes de llegar a Git.

- Husky + lint-staged
- Stack de pre-commit
- Conventional Commits

Husky + lint-staged

La primera línea de defensa ocurre antes de que el código llegue siquiera a Git.



Hooks de Git

Scripts que se ejecutan solos en eventos como commit o push. Husky los versiona dentro del repo.



Solo lo modificado

lint-staged corre el linter únicamente sobre los archivos en staging. Por eso es instantáneo.



Bloqueo estricto

Si el código no cumple el estándar, el commit se cancela. Sin excepciones ni "después lo arreglo".

```
.husky/pre-commit  
  
#!/usr/bin/env sh  
  
. "$(dirname -- "$0")/_/husky.sh"  
  
npx lint-staged
```

```
package.json  
  
"lint-staged": {  
  "*.ts,tsx": ["eslint --fix", "prettier -w"],  
  "*.py": ["ruff check --fix"]  
}
```

Stack de pre-commit recomendado

Tres piezas. Ni una más: el pre-commit que tarda 40 segundos termina desactivado.



Linters & Formatter

ESLint + Prettier en JS/TS, Ruff en Python, o el equivalente de tu lenguaje. Es la base de la limpieza estática.

```
eslint --fix  
prettier --write
```



Husky + lint-staged

El orquestador local. Instala los hooks y decide qué corre sobre qué archivos.

```
"prepare": "husky"  
"pre-commit": "lint-staged"
```



Commitlint

Fuerza Conventional Commits (feat:, fix:). Es lo que después permite generar changelogs solos.

```
commitlint --edit $1
```

Regla de oro: nunca llega código sucio al repositorio.

Conventional Commits en 30 segundos

Un prefijo por commit. A cambio obtenés changelog, versionado semántico y un historial legible.

PREFIJO	SIGNIFICADO	RELEASE
feat:	nueva funcionalidad	minor
fix:	corrección de bug	patch
refactor:	cambio interno sin alterar la API	—
docs:	solo documentación	—
test:	agregar o corregir tests	—
chore:	build, dependencias, config	—

así se ve en la práctica

```
feat(auth): login con OAuth
fix(api): timeout en /users
chore: bump node a 20.x
```



Changelog automático

release-please o semantic-release leen los prefijos y arman el CHANGELOG solos.



Versionado semántico

feat sube minor, fix sube patch, BREAKING CHANGE sube major. Sin discutirlo.



Historial buscable

git log --grep="^fix" te da todos los bugs corregidos del mes.

02

SECCIÓN

CI en la nube

GitHub te regala runners. Usalos para correr lo que no querés correr a mano nunca más.

- Anatomía de Actions
- Nivel 1: unit tests
- Nivel 2: regresión + E2E
- Matriz por rama
- Velocidad y caché
- Secrets y environments

Anatomía de GitHub Actions

Todo pipeline vive en `.github/workflows/*.yml` y se estructura como cajas dentro de cajas.

Workflow

el proceso completo · "CI Pipeline"

Job

pasos que corren en el mismo runner/VM

Step

una tarea individual · instalar deps

Action

comando reutilizable dentro de un step

Triggers: ¿cuándo se ejecuta?

```
.github/workflows/ci.yml

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main", "dev" ]
  workflow_dispatch: # botón manual
  schedule:           # cron
    - cron: '0 2 * * *'
```

Unit tests en cada push y PR

Validación rápida y constante. Si tarda más de dos minutos, la gente deja de esperarla.

```
.github/workflows/unit.yml

name: Unit Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run test:unit
```



Objetivo: < 2 minutos

Feedback dentro del PR, no en tu cabeza. Si tarda más, móvete lo pesado al nivel 2.



npm ci, no npm install

Instala exactamente el lockfile. Reproducible y bastante más rápido en CI.



cache: 'npm'

setup-node cachea node_modules entre corridas. Una línea, minutos ahorrados.

Regresión y E2E hacia main

No querés correr un E2E de 15 minutos en cada commit. Lo reservás para las integraciones que importan.

condicional por rama destino

```
jobs:
  e2e-tests:
    # solo si el PR apunta a main
    if: github.base_ref == 'main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Playwright tests
        run: npx playwright test
```

Ojo: un job con if falso aparece como "skipped", no como fallo. No bloquea el merge.

El arsenal completo



Tests de regresión

Suite completa que asegura que los bugs viejos no reaparecen.



Tests E2E

Cypress o Playwright simulando usuarios reales en un navegador.



Build check

Verifica que la app compile y que el artefacto se genere sin errores.

Matriz de tests por rama destino

Ejecución selectiva para no quemar minutos de Actions en pruebas que no aportan en esa etapa.

FLUJO DE INTEGRACIÓN	LINT	UNIT	REGRESIÓN	E2E
feature/* → dev	✓	✓	—	—
dev → qa / staging	✓	✓	✓	—
qa → main	✓	✓	✓	✓

Rápido donde trabajás

En feature/* solo lint y unit: querés respuesta en segundos, no un informe completo.

Pesado donde duele

Regresión y E2E solo cuando el código se acerca a producción.

Minutos son dinero

Los repos privados pagan por minuto de runner. Esta matriz es también una decisión de costos.

Que el CI no te haga esperar

Un pipeline lento se termina ignorando o desactivando. La velocidad es una feature.

4m 10s

sin caché

52s

con caché de deps

×3

jobs en paralelo

lo que copiás y pegás

concurrency:

group: \${{ github.ref }}

cancel-in-progress: true

jobs:

lint: { runs-on: ubuntu-latest }

unit: { runs-on: ubuntu-latest }

build: { runs-on: ubuntu-latest }

los tres arrancan a la vez

- uses: actions/cache@v4

with:

path: ~/.npm

key: npm-\${{ hashFiles(

'**/package-lock.json') }}



Cacheá dependencias

cache: 'npm' en setup-node, o actions/cache con la clave del lockfile.



Paraleliza jobs

Lint, unit y build no dependen entre sí: que corran a la vez.



Cancelá corridas viejas

concurrency con cancel-in-progress evita gastar runners en commits ya superados.

Secrets y environments

En algún momento el pipeline va a necesitar credenciales. Ese es el momento donde se filtran.



Repository secrets

Settings › Secrets and variables › Actions. Se inyectan como variables y GitHub los enmascara en los logs.



Environments

Agrupá secrets por entorno (staging, production) y exigí aprobación manual antes de desplegar a prod.



Nunca en el repo

Ni .env, ni claves en el YAML, ni tokens en un comentario "temporal". Un repo público indexa en minutos.

uso en el workflow

```
deploy:
  environment: production
  steps:
    - run: ./deploy.sh
    env:
      API_KEY: ${ secrets.API_KEY }
      HOST: ${ vars.HOST }
```

Si ya subiste una clave: **rotala**. Borrarla del historial no la desinvalida.

03

SECCIÓN

Ramas y protección

Las reglas de rama son el contrato que firmás con tu yo de las 2 de la mañana.

- Estándar de ramas
- Protección de ramas
- Por qué proteger main

Estándar de ramas

GitFlow simplificado: cuatro niveles, cada uno con un contrato claro sobre qué puede entrar.



`feature/login-oauth → dev → qa/staging → main`

Protección de ramas

Settings › Branches › Add rule. Cinco minutos de configuración que te ahorran incidentes.



Require a pull request before merging

Prohíbe los commits directos. Todo cambio pasa por un PR, aunque lo abras y lo cierres vos mismo.



Require status checks to pass

El botón de merge se bloquea si las Actions fallan. El código roto no entra, punto.



Require approvals

Al menos una aprobación. Si estás solo, mantené el ritual: revisá tu propio diff antes de mergear.



Do not allow force pushes / deletions

Evita reescribir la historia o borrar la rama principal por accidente.

Todas estas reglas conviven con un repo de una sola persona. Ninguna necesita un segundo humano.

¿Por qué proteger main si estoy solo?

Es tentador saltarse los controles cuando no hay equipo. Estas son las cuatro razones para no hacerlo.

1

Prevención de desastres

Evita el git push --force a main a las 2 AM que te borra tres días de trabajo.

2

Garantía de ejecución

Te asegurás de que los tests corrieron sí o sí antes de que el código toque producción.

3

Auditoría personal

Historial limpio agrupado por PRs con intención clara. Dentro de seis meses lo vas a agradecer.

4

Escalabilidad inmediata

Construís el hábito. Si mañana se suma otro dev, el repo ya está listo para colaborar sin fricción.

`git push origin main --force` · la razón por la que existen las branch protection rules

04

SECCIÓN

Trazabilidad

El código dice qué hace. Los Issues dicen por qué lo hiciste. Esa segunda parte se olvida en tres meses.

- Con Issues vs sin Issues
- El poder de los Issues

Con Issues vs sin Issues

La diferencia no se nota hoy. Se nota cuando volvéis al proyecto después de un trimestre.

El caos

- Commits sueltos: "fix bug", "update api", "cambios".
- Contexto perdido: ¿por qué modifiqué esta validación rara?
- Tareas en post-its o en apps externas no ligadas al código.
- Nadie —ni vos— puede reconstruir la decisión original.

El control

- Cada problema tiene un ID. Cada rama nace de un Issue.
- El PR se vincula solo: Fixes #12 o Closes #45.
- Al hacer merge, GitHub cierra el Issue automáticamente.
- El código incluye la conversación, la intención y el negocio.

```
git checkout -b fix/12-validacion-email → commit: fix(auth): valida email vacío · Fixes #12
```

El poder de los Issues

Cuatro usos concretos que funcionan igual de bien en un repo de una persona.



Descarga mental

Tu backlog integrado al repo. No guardes bugs, ideas ni refactors pendientes en la cabeza.



Issue templates

Plantillas markdown en `.github/ISSUE_TEMPLATE/` que te obligan a describir bien el problema.



Labels reales

bug, enhancement, tech-debt. Un filtro te dice al instante la salud del proyecto.



Milestones

Agrupá Issues por versión (v1.2.0) y mirá el progreso real hacia el próximo release.

Atajo: creá el Issue antes de escribir código. Nombrá la rama con su número. El PR se vincula solo y al mergearse se cierra todo. Cero trabajo administrativo extra.

05

SECCIÓN

Orquestación

Cuando tenés más de un workflow, el orden importa tanto como el contenido.

- Encadenar workflows
- La cadena típica
- Errores comunes

Encadenar workflows

Tres mecanismos distintos. Elegir el correcto evita duplicar YAML en cada repositorio.

needs:

Define trabajos secuenciales dentro del mismo workflow. El job de deploy solo arranca si el de test terminó bien.

```
deploy:
  needs: [test, build]
```

workflow_run:

Dispara un workflow cuando otro termina. Útil para separar lógicas: Release arranca tras CI-Main.

```
on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
```

workflow_call:

Workflows reutilizables. Escribís el deploy una vez y lo llamás desde todos tus microservicios.

```
uses: org/.github/
      .github/workflows/
      deploy.yml@main
```

La cadena típica

Si un eslabón falla, la cadena se detiene ahí. Nada roto llega al final.



el YAML completo de la cadena

```
jobs:
  lint_and_unit: { ... }
  build:         { needs: [lint_and_unit] }
  e2e_tests:     { needs: [build] }
  deploy:        { needs: [e2e_tests] }
```

Falla rápido

lint_and_unit tarda 40s. Si algo está mal, te enterás antes de gastar el runner del build.

Un solo artefacto

El build se genera una vez y se comparte con actions/upload-artifact. Lo que probás es lo que desplegás.

Errores comunes que vas a cometer

Los cuatro que más se repiten cuando alguien monta su primer pipeline.



Un solo workflow gigante

Todo en un job de 300 líneas que tarda 20 minutos.

✓ Separá por responsabilidad y encadená con needs:.



Tests que fallan a veces

Un test flaky enseña al equipo a ignorar el rojo.

✓ Arreglalo o marcalo como skip. Nunca lo dejes intermitente.



continue-on-error en todo

El pipeline queda verde aunque nada haya pasado.

✓ Si un check no debe bloquear, sacalo de los required checks.



Versionar acciones con @main

Una actualización externa te rompe el CI sin que toques nada.

✓ Fijá versión: actions/checkout@v4 o directamente el SHA.

Tu checklist accionable

No intentes montar todo hoy. Tres tandas, en este orden, y cada una ya te deja mejor que ayer.

Hoy

30 minutos

- ☐ Instalar husky, lint-staged y commitlint.
- ☐ Definir qué corre sobre qué archivos.
- ☐ Hacer un commit sucio a propósito y verlo fallar.

Esta semana

2 horas

- ☐ Crear `.github/workflows/ci.yml` con lint + unit.
- ☐ Activar cache y concurrency.
- ☐ Proteger main y dev: PR obligatorio + status checks.

Este mes

según el proyecto

- ☐ Sumar E2E condicional al PR hacia main.
- ☐ Issue templates y labels en `.github/`.
- ☐ Encadenar build → test → deploy con needs:.

Si solo podés hacer una cosa: protegé main y exigí que los status checks pasen.

```
$ git push origin main
```

Dormir tranquilo.

Automatizar hoy es no apagar incendios mañana.

github.com/jhoel0521

docs.github.com/actions

Todo verde

- ✓ pre-commit
- ✓ unit tests
- ✓ e2e
- ✓ branch protection
- ✓ deploy